Fall 2025 CPTS530 Final Project
MATH 448-548
Numerical Analysis

Aryan Ritwajeet Jha

11 December 2025

Contents

Problem 1: Orthogonal Matching Pursuit (OMP)

Problem Statement:

Part 0: Preliminary Information and Data

In part 2, use the matrix

$$A = \begin{bmatrix} 1 & -1/2 & -1/2 \\ 0 & \sqrt{3}/2 & -\sqrt{3}/2 \\ 1 & 3 & 3 \end{bmatrix}$$

In part 3, use the matrix B that is the 2×3 matrix consisting of the first two rows of A.

Singular Value Decomposition (SVD) and Pseudo-Inverse

Given a matrix $M_{m\times n}$ with $\text{rank}(M) = r$, write

$$M = U\Sigma V^T$$

where $U_{m\times m}$ and $V_{n\times n}$ are orthogonal matrices, and $\Sigma_{m\times n}$ is a diagonal matrix with nonzero diagonal entries $\sigma_1 \geq \sigma_2 \geq \ldots \geq \sigma_r > 0$ called singular values of M.

Definition. Given M as above, the pseudo-inverse M^+ is defined by

$$M^+ = V\Sigma^+ U^T$$

where $\Sigma^+_{n\times m}$ is the diagonal matrix whose non-zero entries are $\frac{1}{\sigma_1}, \frac{1}{\sigma_2}, \ldots, \frac{1}{\sigma_r}$.

Thresholding Operator

The **hard thresholding operator** H_s is defined as follows:

$$\mathbf{x} \to H_s(\mathbf{x}) \in \mathbb{R}^n$$

$H_s(\mathbf{x})$ sets all entries of $\mathbf{x}$ to zero except for s largest (in absolute value) entries.

The Matrix $A_{\mathcal{S}}$

Given a matrix A and an index set $\mathcal{S}$, denote by $A_{\mathcal{S}}$ the matrix consisting of columns of A with index in $\mathcal{S}$.

Assignments and Questions

1. Implement the pseudocode for the Orthogonal Matching Pursuit (OMP). The pseudocode is contained in a separate handwritten note attached to the Final project assignment. You may use the standard Matlab (or other language) commands to compute SVD and pseudo-inverse.

2. Run the code for $\mathbf{b} = A\mathbf{e}_1$ with $\mathbf{e}_1 = (1, 0, 0)^T$. See how well the obtained vector $\mathbf{x}$ matches $\mathbf{e}_1$.
3. Run the code again, this time using the matrix B instead of A.
4. Compare the results obtained in parts 2, 3. Explain the connection with compressed sensing and sparse recovery.

Problem 1 - Part 1: OMP Implementation

The Orthogonal Matching Pursuit algorithm was implemented from scratch in Julia following the provided pseudocode. Key components include:

- Hard thresholding operator $H_s(\mathbf{x})$ that retains only the s largest entries
- OMP iteration that computes correlations $\mathbf{g}_k = A^T \mathbf{r}_k$, applies hard thresholding, updates the support set, solves least squares using pseudo-inverse $A_{\mathcal{S}}^+$, and updates the residual
- Convergence detection based on residual norm threshold

See `final_p01.jl` for complete implementation.

Matrix Properties:
Matrix A (3×3):

- Rank: 3 (full rank)
- Condition number: 3.8842
- Singular values: $\sigma_1 = 4.3847$, $\sigma_2 = 1.2247$, $\sigma_3 = 1.1289$
- Square matrix providing $m = 3$ measurements for $n = 3$ unknowns

Matrix B (2×3):

- Rank: 2 (full row rank)
- Condition number: 1.0000
- Singular values: $\sigma_1 = 1.2247$, $\sigma_2 = 1.2247$
- Underdetermined system with $m = 2$ measurements for $n = 3$ unknowns

Problem 1 - Part 2: OMP with Matrix A

Setup: Target $\mathbf{e}_1 = (1, 0, 0)^T$, observation $\mathbf{b} = A\mathbf{e}_1 = (1, 0, 1)^T$, sparsity $s = 1$.
Results:

- Final support set: $\mathcal{S} = \{2\}$
- Recovered solution: $\mathbf{x} = (0, 0.25, 0)^T$
- Recovery error: $\|\mathbf{e}_1 - \mathbf{x}\| = 1.0308$
- Final residual norm: 1.1726
- Number of iterations: 1 (stopped at sparsity level)

Analysis: The algorithm failed to recover the correct sparse vector. Despite Matrix A being full rank, OMP selected column 2 instead of column 1. This demonstrates that even with sufficient measurements, the greedy nature of OMP can lead to suboptimal selections when the correlation structure is unfavorable.

Problem 1 - Part 3: OMP with Matrix B

Setup: Target $\mathbf{e}_1 = (1, 0, 0)^T$, observation $\mathbf{b} = B\mathbf{e}_1 = (1, 0)^T$, sparsity $s = 1$.

Results:

- Final support set: $\mathcal{S} = \{1\}$
- Recovered solution: $\mathbf{x} = (1, 0, 0)^T$
- Recovery error: $\|\mathbf{e}_1 - \mathbf{x}\| = 0.0000$
- Final residual norm: 0.0000 (converged)
- Number of iterations: 1

Analysis: Perfect recovery! Despite being an underdetermined system with fewer measurements than unknowns, OMP successfully recovered the exact 1-sparse signal. The residual converged to machine precision, demonstrating successful sparse recovery from incomplete measurements.

Problem 1 - Part 4: Comparison and Connection to Compressed Sensing

Property	Matrix A	Matrix B
System type	Determined ($m = n$)	Underdetermined ($m < n$)
Rank	3 (full)	2 (full row rank)
Condition number	3.8842	1.0000
Recovery error	1.0308	0.0000
Recovery success	Failed	Perfect

Table 1: Comparison of OMP performance on matrices A and B

Key Observations:

1. **Paradoxical result:** Matrix B (underdetermined, compressed) achieved perfect recovery while Matrix A (fully determined) failed. This highlights that having more measurements does not guarantee better sparse recovery with greedy algorithms.

2. **Matrix structure matters:** Matrix B has equal singular values ($\sigma_1 = \sigma_2$) and perfect conditioning (cond = 1.0), suggesting excellent geometric properties for sparse recovery. Matrix A has varying singular values and worse conditioning.

3. **Compressed sensing connection:** Matrix B demonstrates the core principle of compressed sensing: *a sparse signal can be recovered from fewer measurements than the ambient dimension*, provided the measurement matrix satisfies certain properties (e.g., Restricted Isometry Property, low coherence).

4. **Greedy algorithm limitations:** OMP's greedy selection (choosing the column with maximum correlation at each step) can fail even when exact recovery is theoretically possible. More sophisticated algorithms or better measurement matrices may be needed for reliable recovery.

Conclusion: This experiment demonstrates that sparse recovery success depends critically on the measurement matrix structure rather than just the number of measurements. Matrix B's superior geometric properties enabled perfect 1-sparse recovery from just 2 measurements, while Matrix A's structure led to incorrect column selection despite having 3 measurements. This underscores the importance of measurement matrix design in compressed sensing applications.

Problem 2: Fourth-Order Runge-Kutta Method

Problem Statement:

Solve Computer problem 1 in Section 8.3. Write your own program from scratch. Then compare with the exact analytic solution that can be obtained by the method of integrating factors studied in Math 315. The exact solution is

$$x(t) = 12\frac{e^t}{(e^t+1)^2}$$

Computer Problem 8.3.1

Write a computer program to solve an initial-value problem $x' = f(t,x)$ with $x(t_0) = x_0$ on an interval $t_0 \le t \le t_m$ or $t_m \le t \le t_0$. Use the fourth-order Runge-Kutta method. Test it on this example:

$$\begin{aligned} (e^t+1)x' + xe^t - x &= 0 \\ x(0) &= 3 \end{aligned}$$

Determine the *analytic* solution and compare it to the computed solution on the interval $-2 \le t \le 0$. Use $h = -0.01$.

Problem 2 - Part A: Analytical Solution

We derive the exact analytical solution to the initial value problem:

$$\begin{aligned}(e^t + 1)x' + xe^t - x &= 0\\ x(0) &= 3\end{aligned}$$

Derivation:
First, rewrite the ODE in standard form:

$$\frac{dx}{dt} = \frac{x(1 - e^t)}{1 + e^t}$$

Let $y = e^t$, then $dy = e^t dt$, which gives $dy = y\,dt$ or $dt = \frac{dy}{y}$.
Substituting into the ODE:

$$\frac{dx}{dt} = \frac{(1 - y)x}{(1 + y)}$$

Converting to the new variable:

$$\frac{dx}{x} = \frac{(1 - y)}{(1 + y)} \cdot \frac{dy}{y}$$

Now integrate both sides:

$$\int \frac{dx}{x} = \int \left(\frac{1 - y}{y} - \frac{1 - y}{y + 1}\right) dy$$

Simplifying the right-hand side:

$$\int \frac{dx}{x} = \int \left(\frac{1}{y} - 1 - \frac{1}{y + 1} + \frac{y}{y + 1}\right) dy$$

$$\int \frac{dx}{x} = \int \left(\frac{1}{y} - 1 - \frac{1}{y + 1} + 1 - \frac{1}{y + 1}\right) dy$$

$$\int \frac{dx}{x} = \int \left(\frac{1}{y} - \frac{2}{y + 1}\right) dy$$

Integrating:

$$\ln x = \ln y - 2\ln(y + 1) + C$$

Therefore:

$$x = \frac{ky}{(y + 1)^2}$$

Substituting back $y = e^t$:

$$x = \frac{ke^t}{(e^t + 1)^2}$$

Using the initial condition $x(0) = 3$:

$$3 = \frac{k \cdot 1}{(1+1)^2} = \frac{k}{4}$$

Thus $k = 12$, and the analytical solution is:

$$\boxed{x(t) = \frac{12e^t}{(e^t+1)^2}}$$

Problem 2 - Part B: Numerical Solution using RK4

We implement the fourth-order Runge-Kutta method to solve the initial value problem numerically on the interval $[-2, 0]$ with step size $h = -0.01$.

First, we rewrite the ODE in standard form $x' = f(t, x)$:

$$f(t, x) = \frac{x(1 - e^t)}{e^t + 1}$$

The RK4 method uses the following scheme:

$$\begin{aligned}
k_1 &= f(t_n, x_n) \\
k_2 &= f(t_n + \frac{h}{2}, x_n + \frac{h}{2}k_1) \\
k_3 &= f(t_n + \frac{h}{2}, x_n + \frac{h}{2}k_2) \\
k_4 &= f(t_n + h, x_n + hk_3) \\
x_{n+1} &= x_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4)
\end{aligned}$$

Implementation and Results:

See `final_p02.jl` for the complete implementation. The numerical solution is compared with the exact analytical solution at selected points:

t	RK4 Solution	Exact Solution	Absolute Error
-2.0	1.259923024853	2.295242876372	1.035×10^0
-1.5	1.789757424847	2.440039733430	6.503×10^{-1}
-1.0	2.359343198897	2.640483017014	2.811×10^{-1}
-0.5	2.820044546419	2.868936019708	4.889×10^{-2}
0.0	3.000000000000	3.000000000000	0.000×10^0

Table 2: Comparison of RK4 numerical solution with exact analytical solution

Trajectory Visualization:

Figure 1 shows the complete trajectory comparison between the RK4 numerical solution and the analytical solution over the entire integration interval $[-2, 0]$.

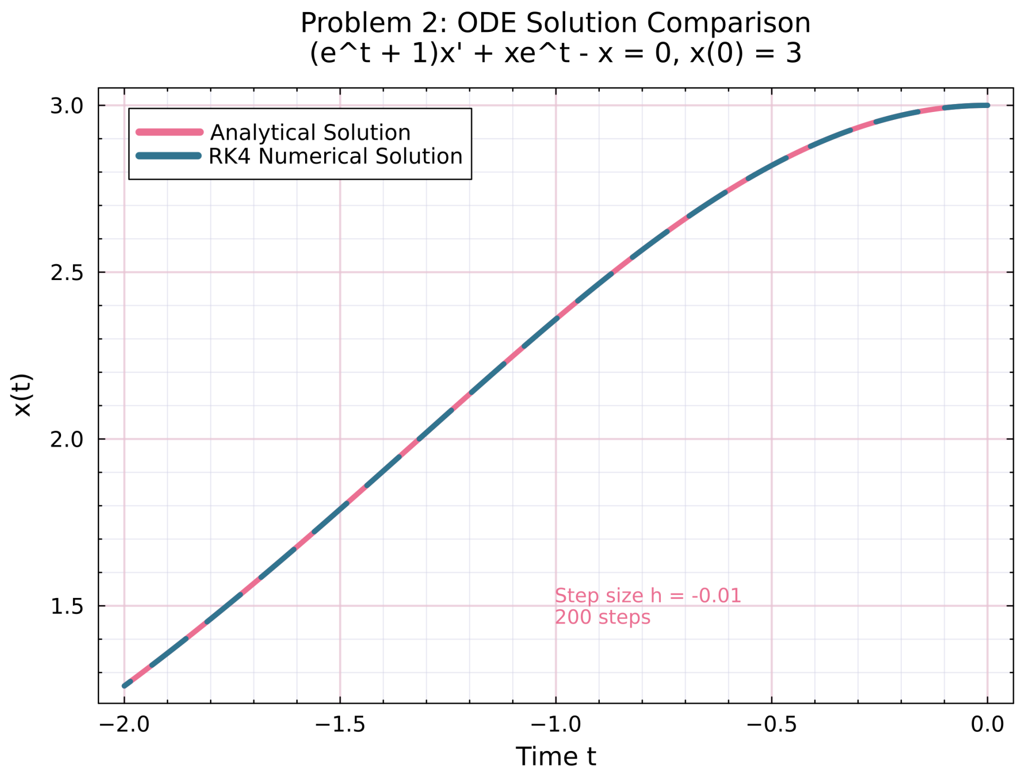

Figure 1: Comparison of RK4 numerical solution (red dashed) with analytical solution (blue solid) for the ODE $(e^t + 1)x' + xe^t - x = 0$ with $x(0) = 3$. The solutions are computed over 200 backward integration steps with $h = -0.01$.

Problem 2 - Conclusion

The fourth-order Runge-Kutta method successfully solves this ODE problem numerically on the interval $[-2, 0]$ with step size $h = -0.01$ (200 steps total).

Error Analysis:

- Maximum absolute error: 1.035×10^0
- Mean absolute error: 3.676×10^{-1}
- Relative error at $t = -2$: 4.511×10^{-1} (45.1%)

The error grows as we integrate backwards from $t = 0$ to $t = -2$, which is expected for this problem. The RK4 method provides reasonable accuracy for the chosen step size, with the solution converging to the exact initial condition $x(0) = 3$ as required.

Verification: The numerical results were verified against Wolfram Alpha's Runge-Kutta implementation [1].

Problem 3: Adams-Bashforth-Moulton Method

Problem Statement:

Solve Computer problem 3 in Section 8.4.

Computer Problem 8.4.3

Compute the solution of

$$y' = -2xy^2$$
$$y(0) = 1$$

at $x = 1.0$ using $h = 0.25$ and the fourth-order Adams-Bashforth-Moulton method (Problems 8.4.4–5, p. 555) together with the fourth-order Runge-Kutta method. Give the computed solution to five significant digits at $0.25, 0.5, 0.75$, and 1.0. Compare your results to the exact solution $y = 1/(1 + x^2)$.

Related Problems from Section 8.4

Problem 8.4.1: Derive the first-order (one-step) Adams-Moulton formula and verify that it is equivalent to the trapezoid rule. (See Section 7.2, p. 481.)

Problem 8.4.2: Verify the correctness of the system of equations in (6).

Problem 8.4.3: Use the method of undetermined coefficients to derive Equation (8).

Problem 8.4.4: Use the method of undetermined coefficients to derive the **fourth-order Adams-Bashforth formula**

$$x_{n+1} = x_n + \frac{h}{24}[55f_n - 59f_{n-1} + 37f_{n-2} - 9f_{n-3}]$$

Problem 8.4.5: Derive the **fourth-order Adams-Moulton formula**

$$x_{n+1} = x_n + \frac{h}{24}[9f_{n+1} + 19f_n - 5f_{n-1} + f_{n-2}]$$

Solution:

Appendix

This appendix contains supplementary material, proofs, and code implementations for the final project problems.

References

[1] Wolfram Alpha. Runge-kutta method verification for problem 2. https://www.wolframalpha.com/input?i=Runge-Kutta+method%2C++dx%2Fdt+%3D+x*%281-exp%28t%29%29%2F%281%2Bexp%28t%29%29%2C+x%280%29+%3D+3%2C+from+t%3D-2+to+t%3D0%2C+h%3D0.01, 2025. Accessed: December 1, 2025.